
```

#ifndef LIST_H
#define LIST_H
/* List.h
 *
 * doubly-linked, double-ended list with Iterator interface
 * EECS 280 Project 4
 */

#include <iostream>
#include <cassert> //assert
#include <cstddef> //NULL

template <typename T>
class List {
    //OVERVIEW: a doubly-linked, double-ended list with Iterator interface
public:
    List(){
        first = nullptr;
        last = nullptr;
    }

    //EFFECTS: Removes all nodes
    ~List(){
        pop_all();
    }

    //EFFECTS: Initializes member variables and copies all nodes from other list
    List(const List<T> &other)
    :first(nullptr), last(nullptr){
        copy_all(other);
    }

    //EFFECTS: Removes all nodes from this list and copies all nodes from other
list
    List& operator= (const List<T> &rhs){
        if (this == &rhs){
            return *this;
        }
        pop_all();
        copy_all(rhs);
        return *this;
    }

    //EFFECTS: returns true if the list is empty
    bool empty() const{
        return (first == nullptr && last == nullptr);
    }

    //EFFECTS: returns the number of elements in this List
    int size() const{
        int count = 0;
        Node *ptr = first;
        while (ptr != nullptr){
            ++count;
            ptr = ptr->next;
        }
        return count;
    }

    //REQUIRES: list is not empty
    //EFFECTS: Returns the first element in the list by reference

```

```

T & front(){
    assert(!empty());
    return first->datum;
}

//REQUIRES: list is not empty
//EFFECTS: Returns the last element in the list by reference
T & back(){
    assert(!empty());
    return last->datum;
}

//MODIFIES: this
//EFFECTS: inserts datum into the front of the list
void push_front(const T &datum){
    Node *ptr = new Node;
    ptr->datum = datum;
    ptr->prev = nullptr;
    if (!empty()){
        ptr->next = first;
        first->prev = ptr;
        first = ptr;
    }
    else{
        ptr->next = first;
        first = ptr;
        last = ptr;
    }
}

//MODIFIES: this
//EFFECTS: inserts datum into the back of the list
void push_back(const T &datum){
    Node *ptr = new Node;
    ptr->datum = datum;
    ptr->next = nullptr;
    if (!empty()){
        ptr->prev = last;
        last->next = ptr;
        last = ptr;
    }
    else{
        ptr->prev = last;
        first = ptr;
        last = ptr;
    }
}

//REQUIRES: list is not empty
//MODIFIES: this, may invalidate list iterators
//EFFECTS: removes the item at the front of the list
void pop_front(){
    assert(!empty());
    Node *victim = first;
    first = victim->next;
    if (size() == 0){
        last = nullptr;
        delete victim;
    }
    else{
        first->prev = nullptr;
        delete victim;
    }
}

```

```

//REQUIRES: list is not empty
//MODIFIES: this, may invalidate list iterators
//EFFECTS: removes the item at the back of the list
void pop_back(){
    assert(!empty());
    Node *victim = last;
    last = victim->prev;
    if (size() == 1){
        first = nullptr;
        delete victim;
    }
    else{
        last->next = nullptr;
        delete victim;
    }
}

// You should add in a default constructor, destructor, copy constructor,
// and overloaded assignment operator, if appropriate. If these operations
// will work correctly without defining these, you can omit them. A user
// of the class must be able to create, copy, assign, and destroy Lists

private:
    //a private type
    struct Node {
        Node *next;
        Node *prev;
        T datum;
    };

    //MODIFIES: this
    //EFFECTS: copies all nodes from other to this
    void copy_all(const List<T> &other){
        for (Node *ptr = other.first; ptr != 0; ptr = ptr->next){
            push_back(ptr->datum);
        }
    }

    //MODIFIES: this, may invalidate list iterators
    //EFFECTS: removes all nodes
    void pop_all(){
        while(!empty()){
            pop_front();
        }
    }

    Node *first;    // points to first Node in list, or nullptr if list is empty
    Node *last;     // points to last Node in list, or nullptr if list is empty

public:
    //////////////////////////////////////
    class Iterator {
        //OVERVIEW: Iterator interface to List

        // You should add in a default constructor, destructor, copy
constructor,
        // and overloaded assignment operator, if appropriate. If these
operations
        // will work correctly without defining these, you can omit them. A user
        // of the class must be able to create, copy, assign, and destroy
Iterators.

        // Your iterator should implement the following public operators: *,

```

```

        // ++ (prefix), default constructor, == and !=.

public:
    Iterator(){
        node_ptr = nullptr;
    }

    // This operator will be used to test your code. Do not modify it.
    // Requires that the current element is dereferenceable.
    Iterator& operator--() {
        assert(node_ptr);
        node_ptr = node_ptr->prev;
        return *this;
    }

    //REQUIRES: this is a dereferenceable iterator
    //EFFECTS: returns the element this operator points to
    T& operator* () const{
        assert(node_ptr != nullptr);
        return node_ptr->datum;
    }

    //REQUIRES: this is a dereferenceable iterator
    //EFFECTS: Increments this iterator to point to the next element. Return
this iterator by reference
    Iterator& operator++ () {
        assert(node_ptr != nullptr);
        node_ptr = node_ptr->next;
        return *this;
    }

    bool operator!= (Iterator rhs) const{
        return node_ptr != rhs.node_ptr;
    }

    //EFFECTS: returns whether this and rhs are pointing to the same place
    bool operator== (Iterator rhs) const{
        return node_ptr == rhs.node_ptr;
    }

private:
    Node *node_ptr; //current Iterator position is a List node
    // add any additional necessary member variables here

    friend class List;

    // construct an Iterator at a specific position
    Iterator(Node *p){
        node_ptr = p;
    }

}; //List::Iterator
////////////////////////////////////

// return an Iterator pointing to the first element
Iterator begin() const {
    return Iterator(first);
}

// return an Iterator pointing to "past the end"
Iterator end() const{
    return Iterator();
}

```

```

//REQUIRES: i is a valid, dereferenceable iterator associated with this list
//MODIFIES: this, may invalidate other list iterators
//EFFECTS: Removes a single element from the list container
// make sure to update size
void erase(Iterator i){
    //assert(i.node_ptr != nullptr);
    if (i == Iterator(first)){
        pop_front();
    }
    else if (i == Iterator(last)){
        pop_back();
    }
    else{
        Node *victim = i.node_ptr;
        i.node_ptr->next->prev = i.node_ptr->prev;
        i.node_ptr->prev->next = i.node_ptr->next;
        delete victim;
    }
}

//REQUIRES: i is a valid iterator associated with this list
//EFFECTS: inserts datum before the element at the specified position.
void insert(Iterator i, const T &datum){
    //assert(i.node_ptr != nullptr);
    if (i == Iterator(first)){
        push_front(datum);
    }
    else if (i == Iterator()){
        push_back(datum);
    }
    else{
        // definitely wrong
        /*
        i.node_ptr = new Node;
        *i.node_ptr = &datum;
        i.node_ptr-- = i.node_ptr;
        i.node_ptr = ++i.node_ptr;
        */
        Node *ptr = new Node;
        ptr->next = i.node_ptr;
        i.node_ptr->prev->next = ptr;
        ptr->prev = i.node_ptr->prev;
        i.node_ptr->prev = ptr;
        ptr->datum = datum;
    }
}

}; //List

```

```

#endif

```

```

////////////////////////////////////
// Add your member function implementations below or in the class above

```

```
// (your choice). Do not change the public interface of List, although you
// may add the Big Three if needed. Do add the public member functions for
// Iterator.
```